

A Personalized Video Game Recommender

Cindy Shih
Stats 418 Project Proposal
2026.05.05



Project Overview & Motivation

Problem

- Players often have limited time but too many games to choose from.
- Most recommendations focus on popularity, rating, or genre.
- They do not always reflect what a player wants to play right now.

Why This Matters

- Game choice depends on play vibe, platform, play style, and time availability.
- A personalized recommender helps users find games that fit the moment.
- Match scores make recommendations easier to compare and interpret.

Goal

Build a personalized video game recommender that suggests games based on play vibe, platform, play style, and session length.

Data Collection Approach

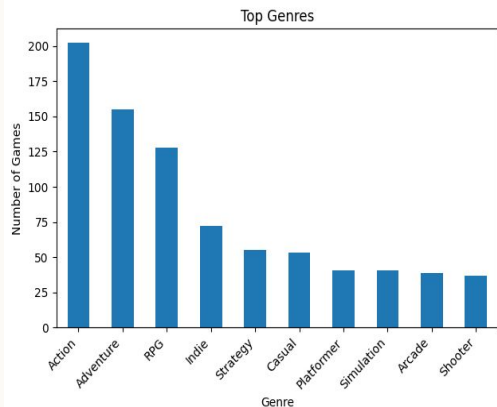


Collect metadata → Clean nested fields → Engineer features for the recommender

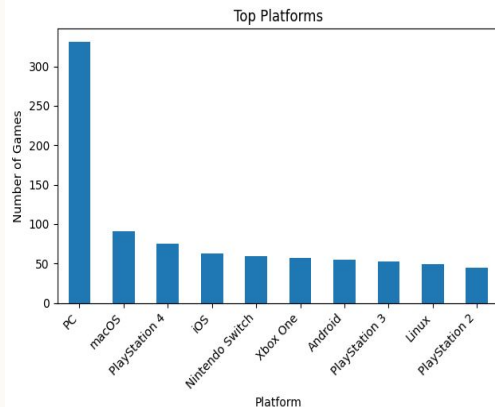
Data Source	API Usage / Process	Current Progress
• RAWG Video Games Database API • Fields: title, genres, tags, platforms, rating, and playtime	• Python API collection script • Cleaning and deduplication • Mapping metadata into app-friendly categories	• API collection script built • Cleaned CSV ready • Initial EDA completed

Exploratory Data Analysis

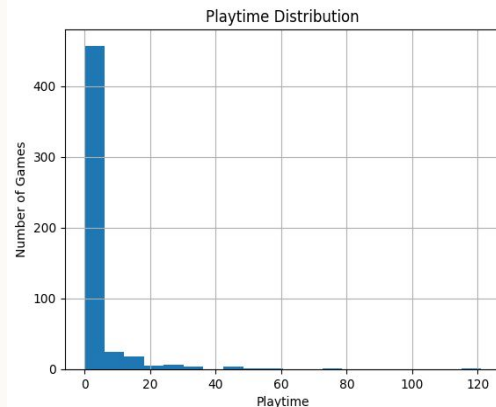
Top Genres



Top Platforms



Playtime Distribution



Key Patterns & Insights

- Action, Adventure, and RPG are the most common genres; PC is the most frequent platform.
- Playtime is highly right-skewed and often missing/unknown, so preprocessing is important.

Data Quality & Preprocessing Needs

Key Data Issue

Playtime = 0 for 336 games (64.62%).
This is likely missing or unavailable metadata, not true zero playtime.

Why It Matters

- **Session-length recommendations depend on playtime.**
- **Zero values can distort distribution-based features.**
- **The app needs user-friendly categories.**

Session Length Mapping

RAWG playtime is calculated in hours.

- **0 hours = Unknown**
- **1 hour = Short**
- **2–8 hours = Medium**
- **More than 8 hours = Long**

Preprocessing Plan

- **Parse nested list columns for genres, tags, and platforms.**
- **Standardize platforms into PC, PlayStation, Xbox, and Switch.**
- **Map genres/tags/descriptions into play vibe categories.**
- **Derive solo vs. multiplayer/co-op play style.**

Proposed Modeling Approach

**User
Inputs**



**Engineered
Features**



**Match Score
(0-100%)**



**Recommended
Game**

User Inputs

- **Play vibe** : Chill/Cozy, Competitive/Intense, Story-Driven, Puzzle/Strategy, Creative/Exploration
- **Platform** : PC, PlayStation, Xbox, Nintendo Switch
- **Play style** : Solo, Multiplayer/Co-op
- **Session length** : Unknown, Short, Medium, Long

Model Type / Logic

- **Baseline**: weighted score / rule-based matching.
- Compare with content-based similarity or k-nearest neighbors if time allows.
- Return match scores and possibly short reasons for each recommendation.

Evaluation Metrics

- **Manual relevance checks** using example user profiles.
- **Coverage** across platforms and play vibes.
- **Diversity** of recommended games.
- **Interpretability** of match reasons.

Timeline & Next Steps

1

Week 6

**Finalize cleaning
& mappings**

2

Week 7

**Complete EDA
& preprocessing**

3

Week 8

**Build scoring model
& prototype**

4

Week 9

**Evaluate, refine
& prepare report**

Potential Challenges

- Many games have missing or zero playtime values.
- Play vibe labels are not directly provided by the API.
- Tags and platform metadata may be noisy or inconsistent.

Mitigation Strategies

- Treat zero playtime as unknown and use fallback rules.
- Use genres, tags, and descriptions to infer play vibe.
- Test with sample user profiles and refine weights.

Current Status

Data collection is complete, and initial EDA has been started.

Remaining work: feature engineering, scoring logic, Streamlit app, evaluation, and report.